SwiftSync $\mathsf{AdHash}_T[H]$

Background

The SwiftSync draft BIP describes an *aggregate* that is the sum of hashes representing elements in a set. Two aggregates are compared at the end of the protocol; if they are equivalent, verification succeeds, otherwise it fails. More formally, the client generates a *salt* and computes $\mathrm{SHA256}_{salt}$ of the coin data, interpreting the digest as an unsigned 256-bit integer.

For a set of coins $A = \{Coin_0, \ldots, Coin_n\}$, define

$$\mathrm{Agg}_A = H_{salt}(Coin_0) + \cdots + H_{salt}(Coin_n).$$

Is it possible to show that this aggregate is provably secure against a probabilistic polynomial-time algorithm? That is, can an attacker create a set B such that $\mathrm{Agg}_A = \mathrm{Agg}_B$ and $A \neq B$?

1 Relation to AdHash

The SwiftSync aggregate is equivalent to $\mathsf{AdHash}_T[H]$ — an instance of the *randomize-then-combine* paradigm [5] — instantiated over $\mathbb{Z}/2^{256}\mathbb{Z}$ with security parameter $\lambda = 256$, a tweakable hash function, a fresh tweak per computation, and without committing to the chunk index.

2 $\mathsf{AdHash}_T[H]$ Definition

Definition 1 ($\mathsf{AdHash}_T[H]$)**.** Let $H : \mathcal{T} \times \mathcal{M} \to \{0,1\}^\lambda$ be a collision-resistant hash function and λ a security parameter. $\mathsf{AdHash}_T[H]$ is defined by the following algorithms.

Setup(1^λ)**:** $k \leftarrow H.\mathrm{Gen}(1^\lambda)$, $p := 2^\lambda$, $\mathcal{I} := \{0,1\}^*$, $\mathcal{D} := \{0,1\}^\lambda$, $\mathcal{T} := \{0,1\}^{|\mathcal{T}|}$. Return $\mathsf{pp} = (k, p, \mathcal{T}, \mathcal{I}, \mathcal{D})$.

Init(pp)**:** $t \leftarrow \mathrm{rand}(\mathcal{T})$, $\quad \mathsf{digest} := 0$.

AddElement(pp, $x \in \mathcal{I}$)**:** $h := H(k, t, x)$, $\quad \mathsf{digest} \leftarrow (\mathsf{digest} + h) \bmod p$, $\quad Q \leftarrow Q \cup \{x\}$.

VerifyHash(pp, Q')**:** Return 1 if $Q' = Q \ \wedge \ \sum_{x \in Q'} H(k, t, x) \bmod p = \mathsf{digest}$, else 0.

3 Correctness

Definition 2 (Perfect Correctness)**.** $\mathsf{AdHash}_T[H]$ is *perfectly correct* if for all $\lambda \in \mathbb{N}$ and all finite sets $Q = Q' \subseteq \{0,1\}^*$:

$$\Pr\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathrm{Setup}(1^\lambda);\ \mathrm{Init}(\mathsf{pp}); \\ \forall\, x \in Q : \mathrm{AddElement}(\mathsf{pp}, x) \end{array} : \mathrm{VerifyHash}(\mathsf{pp}, Q') = 1\right] = 1.$$

Proposition 1. $\mathsf{AdHash}_T[H]$ *is perfectly correct.*

Proof. Assume for contradiction that Proposition 1 is false. Then there exist $\lambda \in \mathbb{N}$ and finite sets $Q = Q' \subseteq \{0,1\}^*$ such that

$$\Pr\left[\begin{array}{l} \mathsf{pp} \leftarrow \mathrm{Setup}(1^\lambda);\ \mathrm{Init}(\mathsf{pp}); \\ \forall\, x \in Q : \mathrm{AddElement}(\mathsf{pp}, x) \end{array} : \mathrm{VerifyHash}(\mathsf{pp}, Q') = 1\right] \neq 1.$$

After $\mathrm{Init}(\mathsf{pp})$ and all calls to $\mathrm{AddElement}(\mathsf{pp}, x)$ for $x \in Q$, the stored digest is

$$\mathsf{digest} = \sum_{x \in Q} H(k,\, t,\, x) \bmod p.$$

When $\mathrm{VerifyHash}(\mathsf{pp}, Q')$ is called, it recomputes

$$\mathsf{digest}' = \sum_{x \in Q'} H(k,\, t,\, x) \bmod p$$

and returns 1 if and only if $\mathsf{digest}' = \mathsf{digest}$. Since $Q' = Q$, the two sums range over the same elements under the same (k, t, p), and H is deterministic. Therefore $\mathsf{digest}' = \mathsf{digest}$ holds with certainty, so VerifyHash returns 1 with probability 1. This contradicts the assumption, and hence Proposition 1 holds. □

4 Security of $\mathsf{AdHash}_T[H]$

Bare AdHash is the most computationally efficient of the three constructions in [1], but Wagner [2] showed a collision attack in time $O(2^{2\sqrt{\lambda}})$ for a λ-bit modulus. Lyubashevsky [3] and Shallue [4] showed the Random Modular Subset Sum problem — equivalent to finding collisions in AdHash — is solvable in $O(2^{\epsilon\lambda})$ for any $\epsilon < 1$.

In $\mathsf{AdHash}_T[H]$, since t is sampled after coins are committed and used only once, does this render these attacks or variants of them inapplicable? What is the concrete bound with security parameter $\lambda = 256$?

Alternatives

- MuHash [1, 6]
- LtHash [5]

References

[1] M. Bellare and D. Micciancio. A New Paradigm for Collision-Free Hashing. *EUROCRYPT 1997*. `https://eprint.iacr.org/1997/001`

[2] D. Wagner. A Generalized Birthday Problem. *CRYPTO 2002*. `https://www.iacr.org/archive/crypto2002/24420288/24420288.pdf`

[3] V. Lyubashevsky. On Random High Density Subset Sums. *ECCC TR05-007*, 2005. `https://eccc.weizmann.ac.il/report/2005/007/`

[4] A. Shallue. An Improved Multi-set Algorithm for the Dense Subset Sum Problem. *ANTS-VIII*, Springer, 2009. `https://link.springer.com/chapter/10.1007/978-3-540-79456-1_28`

[5] K. Lewi, W. Kim, I. Maykov, and S. Weis. Securing Update Propagation with Homomorphic Hashing. *IACR ePrint 2019/227*. `https://eprint.iacr.org/2019/227`

[6] P. Wuille. Rolling UTXO Set Hashes. *Bitcoin-dev*, 2017. `https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2017-May/014337.html`

[7] P. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms. *SIAM J. Comput.*, 1997. https://arxiv.org/abs/quant-ph/9508027

[8] L. Grover. A Fast Quantum Mechanical Algorithm for Database Search. *STOC 1996*. https://arxiv.org/abs/quant-ph/9605043

[9] V. Lyubashevsky, C. Peikert, and O. Regev. On Ideal Lattices and Learning with Errors Over Rings. *EUROCRYPT 2010*. https://link.springer.com/chapter/10.1007/978-3-642-13190-5_1

[10] NIST. Post-Quantum Cryptography Standards, FIPS 203–205, 2024.